

# ProvGuard: Logic-Aware Multi-View Contrastive Learning for Robust and Efficient Host Threat Detection

Anyuan Sang  
Xidian University  
Xi'an, China  
anyuan@stu.xidian.edu.cn

Li Yang  
Xidian University  
Shaanxi Key Laboratory of Network  
and System Security  
Xi'an, China  
yangli@xidian.edu.cn

Lu Zhou\*

Xidian University  
Xi'an, China  
zhoulu@xidian.edu.cn

Cheng Zhou  
Xidian University  
Xi'an, China  
chengzhou@stu.xidian.edu.cn

Junbo Jia  
Xidian University  
Xi'an, China  
jbjia@stu.xidian.edu.cn

Huipeng Yang  
Xidian University  
Xi'an, China  
yanghp@stu.xidian.edu.cn

## Abstract

The security of web services increasingly relies on accurate detection of advanced, previously unseen attacks hidden within complex host activities. Provenance-based intrusion detection systems (PIDSeS) offer a promising foundation for this task by capturing rich causal and structural relationships across processes, files, and network interactions. However, recent studies show that these graph-driven methods remain vulnerable to graph manipulation attacks, where adversaries subtly alter provenance graphs to evade detection, which limits their practical deployment.

To address this challenge, we present ProvGuard, a robust anomaly detection framework that couples logic-aware multi-view augmentation with contrastive representation learning. Instead of applying arbitrary structural perturbations, ProvGuard employs Logic-Aware Noise Injection (LNI) to generate semantically valid graph views that preserve the causal semantics of provenance data. These views are then leveraged in a Logic-Preserving Contrastive Learning module, enabling the model to learn representations invariant to benign transformations yet sensitive to adversarial inconsistencies. Extensive evaluations on multiple provenance datasets show that ProvGuard surpasses state-of-the-art detectors in resisting graph manipulation attacks while maintaining high detection accuracy and efficiency, achieving an average F1-score above 96% with less than a 10% AUC drop.

## CCS Concepts

• **Security and privacy** → **Intrusion detection systems**; • **Computing methodologies** → *Machine learning*.

## Keywords

System Security, Intrusion Detection Systems, Provenance Graph

## ACM Reference Format:

Anyuan Sang, Li Yang, Lu Zhou, Cheng Zhou, Junbo Jia, and Huipeng Yang. 2026. ProvGuard: Logic-Aware Multi-View Contrastive Learning for Robust and Efficient Host Threat Detection. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792269>

## 1 Introduction

Advanced Persistent Threats (APTs) have become increasingly prevalent, posing significant risks to global cybersecurity [1]. These sophisticated and stealthy attacks target critical infrastructure, government systems, and private enterprises, often leading to severe data breaches, financial losses, and national security threats. The persistent nature of APTs allows attackers to maintain a foothold within compromised networks for extended periods, enabling them to exfiltrate sensitive information and disrupt operations, causing widespread harm to society and the economy.

Provenance graphs record the causal relationships between system entities and events and form the foundation of provenance-based intrusion detection systems (PIDS). By leveraging the rich context encoded in these graphs, PIDS enables in-depth analysis of system activities and potential attack chains. Recently, learning-based detection methods have been widely adopted. In the upstream graph learning stage, they learn node embeddings [10, 26, 43, 44, 55], edge embeddings [54, 60], and subgraph embeddings [48, 58] to obtain expressive representations of provenance graphs. These representations capture both contextual and structural information. In the downstream stage, detectors apply anomaly detection and vector similarity analysis to identify anomalies.

While PIDS have achieved notable detection accuracy, emerging *graph manipulation attacks* reveal inherent weaknesses that current defenses cannot effectively mitigate. Such attacks deliberately forge provenance relationships—for example, inserting benign-looking edges to connect malicious processes with clean nodes—thereby shifting malicious subgraphs in the embedding space and allowing them to evade detection [15, 39, 46]. This vulnerability reflects the intrinsic sensitivity of graph-based learning models, where even minor structural perturbations can lead to high-confidence misclassification [6, 11]. Although a few recent studies have examined the

\*Lu Zhou is the Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*  
© 2026 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-2307-0/2026/04  
<https://doi.org/10.1145/3774904.3792269>

impact of graph manipulation in provenance graphs [19, 26, 43, 55], targeted defenses remain largely unexplored.

Existing approaches therefore suffer from three interrelated limitations: (1) **Scarcity of high-quality malicious data**, where the limited availability of labeled attack data hampers the use of generic robustness techniques such as adversarial training [4, 53]; (2) **Inefficiency of current defenses**, as some methods attempt to enhance robustness by employing increasingly complex graph neural networks to capture deeper structural features, but achieve only modest gains while incurring substantial computational and memory overhead [43, 44]; and (3) **Sensitivity of graph neural architectures**, since GNN-based detectors remain highly vulnerable to subtle structural changes, allowing carefully crafted edge insertions to distort node embeddings and mislead downstream detection [6, 11]. These limitations highlight the need for a new robustness-enhancement framework capable of learning representations that are invariant to benign-appearing perturbations while remaining sensitive to global malicious semantics.

In this paper, we present **ProvGuard**, a provenance-graph anomaly detector that achieves strong robustness against *graph manipulation attacks* while maintaining high detection accuracy. Such attacks mimic benign behaviors by inserting deceptive edges so that malicious subgraphs shift in the embedding space and evade detection [15, 39, 46]. We analyze the PIDS workflow and identify two major attack types: graph poisoning during training and graph pollution during detection. To counter them, ProvGuard integrates *logic-aware multi-view graph augmentation* with *contrastive learning*, forcing representations to remain invariant to local perturbations yet sensitive to global malicious semantics.

The augmentation enforces provenance constraints (e.g., disallowing invalid file–network edges), generating realistic adversarial variants and guiding the model to focus on stable graph-level patterns. Contrastive learning then aligns embeddings of these views while separating unrelated behaviors, emphasizing semantic consistency beyond superficial structural similarity [23, 59]. This design yields strong resistance to both poisoning and evasion attacks and improves generalization in adversarial environments. After representation learning, ProvGuard performs unsupervised detection using lightweight K-Means clustering. Cluster centroids and average intra-cluster distances from training serve as references; at inference, a node’s minimum distance to these centroids provides a normalized anomaly score.

To comprehensively evaluate the performance of ProvGuard, we used five commonly adopted provenance datasets: DARPA TC THEIA, CADETS, TRACE [3], the Streamspot dataset [2], and the Unicorn Wget dataset [18]. We selected five *state-of-the-art* (SOTA) graph learning-based anomaly detection methods as baselines: **Unicorn** [18], **Threatrace** [55], **MAGIC** [26], **FLASH** [43], and **STGAN** [44]. To rigorously test the robustness of ProvGuard against graph manipulation attacks, we implemented five different attack types in both the detection and training phases following prior evasion studies [15, 39, 46]. Experimental results show that ProvGuard achieves the best overall performance with an F1 score above 96% and maintains strong robustness with an AUC drop of less than 10%. We also conducted ablation studies to validate the design and effectiveness of each component of ProvGuard.

Our contributions are summarized as follows:

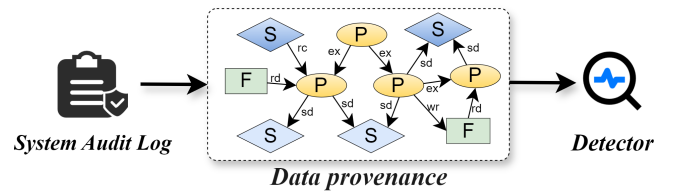
- **Robustness Against Graph Manipulation Attacks:** We are the first to investigate adversarial attacks in PIDS and propose the first PIDS framework specifically designed to defend against graph manipulation attacks.
- **Logic-Aware Multi-View Contrastive Learning:** We introduce a logic-aware multi-view contrastive strategy that enforces semantic consistency and delivers strong robustness without heavy model complexity.
- **Extensive evaluation:** We built a prototype of ProvGuard and conducted a comprehensive evaluation on five datasets. The experiments demonstrated that ProvGuard’s detection performance and robustness surpass five *SOTA* methods, including **Unicorn** [18], **Threatrace** [55], **MAGIC** [26], **FLASH** [43], and **STGAN** [44].

**Ethics and Privacy:** All datasets are sourced from public websites<sup>1</sup>, and experiments were conducted in a controlled environment to minimize potential ethical and privacy risks.

## 2 Background

### 2.1 Provenance-based IDS

Since the provenance graph can express the relationship between system operating entities in time, existing research has used this feature to build an IDS based on the provenance graph. Including detection schemes based on knowledge labels [20, 24, 38, 63], these schemes construct a series of matching rules based on expert knowledge to match in the origin graph to detect anomalies. Based on the statistics IDS scheme [14, 21, 34], they use the structural feature information of the graph, including: abnormality, discrepancy, time correlation, and other features to analyze in the graph to detect anomalies. Recently, more learning-based IDS solutions have been proposed [5, 19, 26, 48, 58, 60]. These solutions use models such as graph representation learning and sequence learning to extract high-dimensional features from graphs to perform anomaly detection in downstream tasks.



**Figure 1: The classical workflow of current PIDS.** F means file node, S means socket node, and P means process node. sd means send, wr means write, ex means exec, rd means read, rc means recv.

As shown in Fig. 1. A typical PIDS pipeline includes (i) real-time collection and storage of audit events, (ii) construction of provenance graphs, and (iii) detection and investigation using graph mining or machine-learning techniques. Prior research has explored rule-based reasoning [24, 38, 45, 63], statistical anomaly detection [13, 18, 21, 36], and graph learning-based methods such as graph neural networks [8, 10, 26, 27, 43, 44]. These approaches have

<sup>1</sup>ProvGuard is available at <https://github.com/anyuan1999/ProvGuard>

demonstrated strong capabilities in uncovering stealthy attacks, lateral movement, and data exfiltration even when conventional Signature-based defenses fail.

## 2.2 Graph Manipulation Attacks

Graph manipulation attacks [15, 39, 46] strategically alter graph structures to evade detection or degrade model performance, posing a significant challenge to graph learning systems. These attacks typically target critical graph elements such as nodes, edges, or features to disguise malicious behavior as benign or to disrupt the learning process of graph-based models. For example, attackers may inject fake nodes or edges to mimic benign entities, thereby increasing the difficulty of anomaly detection. To better define this category of attacks, we analyze the classical workflow of PIDS as illustrated in Figure 2 and identify two primary types of attacks: poisoning attacks during the training phase and data contamination attacks during the detection phase.

## 2.3 Threat Model

We consider an enterprise or cloud host environment where a PIDS continuously collects fine-grained system audit logs, process creation, file and network events, and inter-process communication to construct a dynamic provenance graph. The defender controls the monitored hosts and assumes that the operating system’s audit framework and provenance recorder are trustworthy and tamper-resistant [7, 22, 30–32, 35, 40, 41, 61]. Audit data are transmitted and stored through standard cryptographically protected channels. In addition, we do not consider hardware side-channel attacks or denial-of-service attacks against the logging infrastructure.

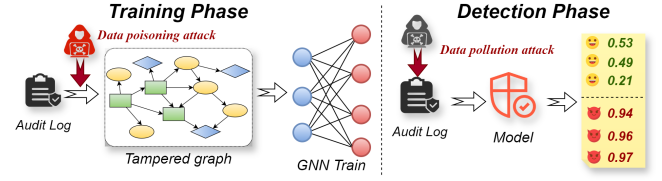
For the adversary, we assume the attacker can compromise the target host in various ways and employ stealth techniques to evade the intrusion detection system, but cannot tamper with the host’s audit subsystem. We further assume the attacker can use scripts to introduce specific perturbations into the provenance graph in order to launch graph manipulation attacks against the PIDS [46].

## 3 Methodology

As shown in Fig. 3, ProvGuard consists of three components: (1) Graph Builder, (2) Graph Representation, and (3) Anomaly Detection. The graph builder converts system audit logs into a provenance graph whose nodes denote system entities and edges denote interactions; redundant structures are compressed for efficiency. The representation module encodes node and edge types, applies a Graph Attention Network (GAT) to capture structural and semantic dependencies, and performs logic-aware graph augmentation with contrastive learning to learn perturbation-invariant yet semantics-sensitive embeddings. Finally, the anomaly detector uses K-Means clustering to score nodes by their distance to cluster centroids.

### 3.1 Graph Builder

Our system accepts streaming system audit logs and constructs the provenance graph, similar to previous research [28, 29]. It consists of three main components. First, ProvGuard streams and extracts audit logs in batches from existing operating systems, such as Windows ETW logs or Linux audit logs. These logs contain information



**Figure 2: The classic detection processes of PIDS identified two types of graph manipulation attacks: data poisoning attacks during the training phase and data pollution attacks during the detection phase.**

about interactions between system entities, including files, processes, and networks. Next, ProvGuard extracts and processes this log information. Specifically, for each audit log within a batch, it extracts the fundamental components representing the nodes and edges of the provenance graph: the quadruple (*src*, *dst*, *timestamp*, *edge type*), where *src* denotes the process node, *dst* represents the file or network node, *timestamp* indicates the time when the event occurred, and *edge type* specifies the type of edge. Finally, to accelerate model training and reduce computational complexity, we adopt multi-class graph denoising techniques from prior studies, removing only redundant nodes and those irrelevant to attack detection. ProvGuard utilizes the CPR (Causal Persistent Reduction) method [57] for edge processing, retaining only one instance of edges that appear multiple times between two nodes within a short time window. Additionally, during graph construction, orphaned nodes and faulty nodes (potentially generated by logging errors) that are unrelated to the attack investigation are removed.

## 3.2 Graph Representation

Graph representation in ProvGuard follows a structured pipeline to produce embeddings robust to anomalies. It begins with feature encoding, where node and edge types are converted into categorical or binary vectors. The raw provenance graph is then stochastically augmented to create two perturbed views under simple semantic constraints. Each augmented view is subsequently encoded by a Graph Attention Network (GAT) to capture local and global dependencies. Finally, a contrastive learning module aligns embeddings across the paired views while maintaining separation between distinct graphs, yielding expressive and resilient representations for reliable detection in complex environments.

**3.2.1 Feature Encoding.** ProvGuard begins by encoding the raw provenance graph’s node and edge attributes using one-hot feature vectors. Each node and edge is represented by its type, and discrete attributes such as node types (e.g., processes, files) and edge types (e.g., read, write) are transformed into binary vectors that preserve their distinct semantics.

**3.2.2 Logic-Aware Graph Augmentation.** ProvGuard employs logic-aware graph augmentation strategies to simulate potential adversarial attacks while maintaining the semantic plausibility of the provenance graph. These augmentations include edge augmentation (EA), node augmentation (NA), and feature augmentation (FA), each applied to generate perturbed views of the original graph. The intensity of each augmentation operation is controlled by the

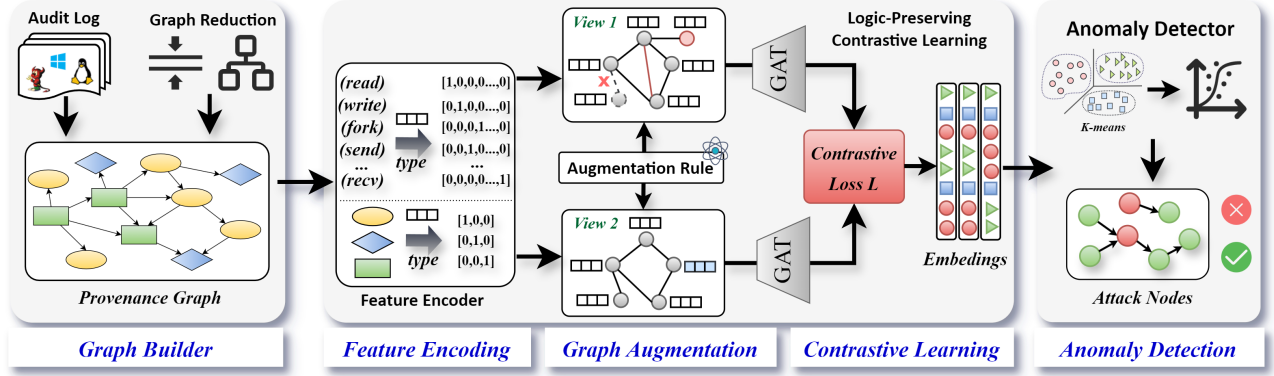


Figure 3: Overview of ProvGuard's architecture.

hyperparameter  $\gamma$ , which specifies the proportion of nodes or edges to be modified.

To ensure the realism and logical validity of augmented graphs, we implement a strategy called **Logic-Aware Noise Injection (LNI)** as shown in Table 1. This strategy enforces rationality constraints during augmentation to prevent the generation of semantically invalid structures. For instance, in edge augmentation, we prohibit the addition of edges that directly connect two network nodes or connect a file node to a network node—such configurations violate causal semantics in provenance graphs.

Table 1: Logic-Aware Edge Augmentation Rules in Provenance Graphs

| Source  | Destination | Allowed | Edge                    |
|---------|-------------|---------|-------------------------|
| Process | File        | ✓       | read/write              |
| Process | Network     | ✓       | connect/send/recv       |
| File    | Process     | ✓       | exec/load               |
| Process | Process     | ✓       | fork/clone              |
| Network | Process     | ×       | violates causality      |
| File    | Network     | ×       | no direct communication |
| Network | File        | ×       | no direct communication |
| Network | Network     | ×       | meaningless edge        |

**Edge Augmentation (EA).** Edge augmentation modifies the graph structure by adding or removing edges under logic constraints. The edge set  $E'$  is modified as follows:

$$E' = E \cup \{(u, v)\},$$

This operation adds an edge between nodes  $u$  and  $v$  only if it does not already exist *and* the connection satisfies domain-specific logic rules.

$$E' = E \setminus \{(u, v)\},$$

Alternatively, an existing edge can be removed. The perturbation intensity is controlled by  $\gamma$ ; for example, 20% of edges are randomly selected for addition or removal, subject to logical validity.

**Node Augmentation (NA).** Node augmentation involves adding or removing nodes along with their associated edges, while ensuring logical consistency in their insertion or removal context:

$$V' = V \cup \{v'\},$$

This means a new node  $v'$  is added to the graph and linked to others only through permissible edge types.

$$V' = V \setminus \{v\}, \quad E' = E \setminus \{(u, v)\},$$

An existing node  $v$  may be removed along with all its connected edges. Node augmentation is also governed by  $\gamma$ ; for example, 20% of the nodes are selected for addition or deletion.

**Feature Augmentation (FA).** Feature augmentation modifies node attributes while preserving semantic alignment. The feature vector of node  $v$  is replaced with that of another node  $w$  of the same type:

$$X'_v = X_w, \quad w \sim \{u \in V \mid \text{type}(u) = \text{type}(v)\},$$

This simulates adversarial feature manipulation without disrupting node-type semantics.

Collectively, these logic-aware augmentations enhance the model's ability to learn invariant patterns and detect adversarial perturbations that preserve surface semantics but violate causal consistency. They prepare the model for challenging attack scenarios while retaining the integrity of the graph's structural and semantic foundations.

**3.2.3 Logic-Preserving Contrastive Learning.** After the augmentation step, ProvGuard employs a Graph Attention Network (GAT) to extract higher-order structural and semantic features from each augmented view. The GNN aggregates information from neighboring nodes and edges, capturing both local dependencies and global contextual patterns. For a node  $v$ , its feature representation  $h_v^{(l+1)}$  at layer  $l+1$  is computed as:

$$h_v^{(l+1)} = \sigma \left( W^{(l)} \cdot \text{AGG} \left( \{h_u^{(l)} \mid u \in \mathcal{N}(v)\} \right) + b^{(l)} \right),$$

where  $h_v^{(l)}$  is the feature vector of node  $v$  at layer  $l$ ,  $\mathcal{N}(v)$  represents the neighbors of  $v$ ,  $\text{AGG}(\cdot)$  is an aggregation function (e.g., sum or mean),  $W^{(l)}$  and  $b^{(l)}$  are learnable parameters, and  $\sigma$  is an activation function (e.g., ReLU).

To further improve robustness, ProvGuard introduces a contrastive learning framework tailored to provenance graphs, with an emphasis on preserving the underlying causal semantics. Unlike general-purpose contrastive learning frameworks such as GraphCL [59] or MVGRL [23], which rely on random augmentations and structure-based similarity, our approach incorporates domain-aware augmentations and logic consistency.

Given two augmented views  $G_i$  and  $G_j$  of the same original graph  $G$ , the encoder generates graph-level embeddings  $z_i$  and  $z_j$ , which are passed through a two-layer projection head:

$$p_v = \text{ReLU}(W_p^{(1)} z_v + b_p^{(1)}), \quad \hat{p}_v = W_p^{(2)} p_v + b_p^{(2)}.$$

We then compute the contrastive loss:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(\hat{p}_i, \hat{p}_j)/\tau)}{\sum_{k=1}^N \exp(\text{sim}(\hat{p}_i, \hat{p}_k)/\tau)},$$

where  $\text{sim}(\cdot, \cdot)$  denotes cosine similarity, and  $\tau$  is a temperature parameter. Positive pairs  $(\hat{p}_i, \hat{p}_j)$  originate from different views of the same graph that preserve logical structure, while negatives  $\hat{p}_k$  come from unrelated graphs.

This design ensures that the learned embeddings reflect consistent high-level behaviors rather than superficial structural features. It enables the model to resist manipulation that mimics graph topology while violating semantic logic, which is especially critical for provenance-based anomaly detection.

### 3.3 Anomaly Detection

To determine the most effective self-supervised anomaly detection mechanism, we evaluated several candidate classifiers, including Local Outlier Factor (LOF) [9], One-Class SVM [47], KMeans [16], and Isolation Forest [33]. As detailed in Section 4.5, KMeans demonstrated superior performance in terms of detection accuracy. Consequently, ProvGuard employs a KMeans-based anomaly detector, which includes training and detection phases.

In our method, we apply K-means to partition the embedding space into  $k$  clusters and retain all cluster centroids for subsequent anomaly detection.

During the detection phase, each new embedding vector is evaluated by computing its distance to the nearest cluster centroid. The anomaly score  $S_i$  for a data point  $x_i$  is defined as the Euclidean distance to the closest centroid among the  $k$  clusters:

$$S_i = \min_{j=1, \dots, k} \|x_i - c_j\|,$$

where  $c_j$  denotes the  $j$ -th cluster centroid obtained from training.

To ensure comparability across datasets and feature scales, we normalize the raw anomaly score using the mean nearest-centroid distance computed on the training set, denoted as  $D_{\text{mean}}$ . This is defined as:

$$D_{\text{mean}} = \frac{1}{N} \sum_{i=1}^N \min_{j=1, \dots, k} \|x_i - c_j\|,$$

where  $N$  is the number of training samples. The normalized anomaly score  $\tilde{S}_i$  is given by:

$$\tilde{S}_i = \frac{S_i}{D_{\text{mean}}}$$

A data point is considered anomalous if its normalized score exceeds a predefined threshold  $\theta$ :

$$\text{Anomaly}(x_i) = \begin{cases} 1 & \text{if } \tilde{S}_i > \theta, \\ 0 & \text{otherwise.} \end{cases}$$

Given the large-scale nature of provenance data, this centroid-based evaluation strategy significantly reduces the inference overhead compared to pairwise distance-based approaches such as KNN [26], while maintaining effective anomaly detection performance, especially for large-scale provenance data.

## 4 Evaluation

### 4.1 Experiments Setup

**Datasets.** We evaluated the performance of ProvGuard under three open-source datasets: DARPA Engagement E3 [3], Streamspot [2], and Unicorn Wget [18]. All three datasets are inconsistent in the scenarios they target and the granularity of their detections, and thus we believe they are able to provide insights into the performance of the system. The details of the dataset description are in the appendix A.

**Baselines.** To comprehensively evaluate the detection performance of ProvGuard, we compare it with five state-of-the-art (SOTA) and open-source methods in the PIDS domain, including Unicorn [18], Threatrace [55], MAGIC [26], FLASH [43], and STGAN [44].

**Graph manipulation attack.** We provide a detailed description of the experimental setup used to evaluate the robustness of ProvGuard against graph manipulation attacks, which aim to evade detection by modifying either the graph structure or node features. We broadly categorize these attacks into two types: data pollution attacks that occur during the detection phase, and data poisoning attacks that take place during the model training phase. Following prior work [10, 19, 26, 43], we adopt five different attack scenarios for comprehensive evaluation (GSPA, GFPA, CGPA, SPA, and FPA). The details of the attack description are in the appendix A.

**Metrics.** We use a variety of common metrics to comprehensively assess the model's behavior under different tasks and experimental setups. The basic evaluation metrics include Recall (Rec), Precision (Pre), F1 Score (F1), Area Under the Curve (AUC), False Positive Rate (FPR), and Accuracy (Acc). Each experiment was run five times and the reported results are the averages.

### 4.2 ProvGuard's Effectiveness

The experiments are conducted on the Unicorn and Streamspot datasets for graph-level detection and the DARPA dataset for node-level detection.

**Detection result.** Table 2 provides the detection results of ProvGuard. In the graph-level anomaly detection datasets, Streamspot and Unicorn wget, ProvGuard achieved near-perfect detection performance on the simpler Streamspot dataset, with a precision of 99% and a recall of 100%. This high performance is attributed to the dataset's collection of single-user activities per log batch, which are structurally and semantically distinct from each other.

On the more complex Unicorn Wget dataset, ProvGuard still achieved high accuracy (98%) and recall (96%). Moving to node-level detection, ProvGuard also demonstrated high performance on

the DARPA TC datasets, achieving 99% accuracy and 99% recall. Due to the significant disparity between benign and malicious entities, ProvGuard was able to identify anomalies accurately. This success is attributed to the use of KMeans for outlier detection, which effectively leverages the distinct feature distributions of benign and malicious entities.

**Comparison study.** To compare the performance of ProvGuard with existing state-of-the-art methods, we selected several detectors for both graph-level and node-level anomaly detection, including Threatrace [55], FLASH [43], Unicorn [18], STGAN [44], and MAGIC [26].

As shown in Table 2, ProvGuard demonstrates outstanding performance across five representative provenance datasets. For node-level detection, it achieves near-perfect results on Theia, Cadets, and Trace. On Theia, ProvGuard reaches 0.9967 in F1-score, while achieving the lowest FPR of 0.03%. On the Cadets and Trace datasets, it further reduces the FPR to below 0.01%, outperforming both supervised detectors like Threatrace, as well as advanced unsupervised methods like MAGIC, FLASH, and STGAN. For graph-level detection, ProvGuard also shows strong performance on both the Streamspot and Wget datasets. On Streamspot, it achieves a perfect recall of 1.00 and a balanced F1-score of 0.99. On the more complex Wget dataset, ProvGuard leads all baselines with an FPR of only 0.6%, significantly outperforming MAGIC and FLASH and especially Threatrace. These results highlight the effectiveness of ProvGuard’s contrastive learning framework, which leverages multi-view graph augmentation to generate robust and generalizable representations. Unlike MAGIC’s masked graph autoencoder approach that focuses on local reconstruction, ProvGuard captures both local and global semantics by contrasting positive and negative views. This enables more reliable identification of stealthy or structurally evasive attack behaviors embedded in the provenance graph.

### 4.3 Adversarial Robustness Analysis

In this section, we conducted a comprehensive and in-depth evaluation of the robustness of ProvGuard against graph tampering attacks. Table 3 presents a comparative analysis of various attack types under different perturbation ratios. Threatrace and FLASH exhibit significant performance degradation under structural perturbations, especially at a 50% attack ratio, where their F1 scores drop to 0.489 and 0.657, respectively. This indicates their strong reliance on local neighborhood structures, making them vulnerable to shifts in the embedding space caused by adversarial modifications. MAGIC, which adopts a node-masking strategy, shows improved local robustness and performs reasonably well under low-intensity attacks (0.8 F1 under GSPA=20%). However, its performance deteriorates notably under structurally intensive global perturbations (i.e., higher attack ratios), revealing its lack of explicit global structural consistency enforcement. STGAN, which leverages spatiotemporal feature fusion and deep feature extraction, achieves more stable performance than previous baselines under both local and global perturbations. Nevertheless, its robustness remains slightly inferior to ProvGuard, particularly under high-intensity structural manipulations. In contrast, ProvGuard, enhanced by multi-view perturbation strategies across nodes, edges, and features, consistently outperforms all baselines across all attack types and ratios.

**Table 2: Comparison Experiments. (Bold denotes the best result, and underlined denotes the second-best result.)**

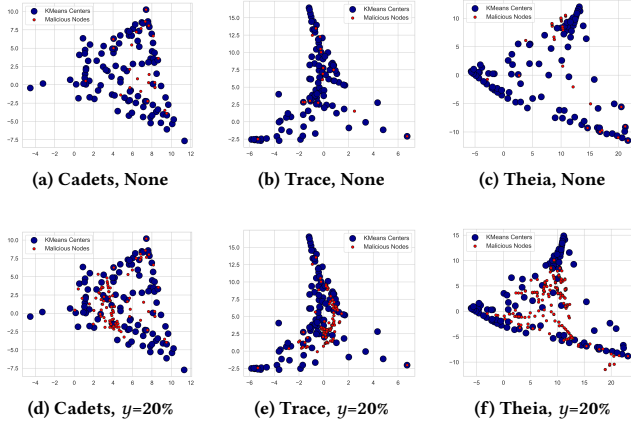
| Dataset    | Method           | Metrics       |               |               |               |              |
|------------|------------------|---------------|---------------|---------------|---------------|--------------|
|            |                  | Accuracy      | Precision     | Recall        | F1-Score      | FPR          |
| Theia      | Threatrace       | <u>0.9987</u> | 0.8703        | 0.9974        | 0.9295        | 0.10%        |
|            | MAGIC            | 0.9986        | <u>0.9823</u> | <b>0.9999</b> | <u>0.9910</u> | 0.14%        |
|            | FLASH            | 0.9938        | 0.9173        | 0.9982        | <u>0.9563</u> | <u>0.8%</u>  |
|            | STGAN            | 0.9959        | 0.9449        | <u>0.9983</u> | 0.9709        | 0.10%        |
|            | <b>ProvGuard</b> | <b>0.9995</b> | <u>0.9955</u> | 0.9978        | <b>0.9967</b> | <b>0.03%</b> |
| Cadets     | Threatrace       | 0.9981        | 0.9042        | <b>0.9997</b> | 0.9495        | 0.20%        |
|            | MAGIC            | 0.9979        | 0.9457        | 0.9976        | 0.9710        | 0.21%        |
|            | FLASH            | 0.9973        | 0.9343        | <u>0.9994</u> | 0.9658        | 0.3%         |
|            | STGAN            | <u>0.9994</u> | <u>0.9840</u> | 0.9983        | <u>0.9911</u> | <u>0.02%</u> |
|            | <b>ProvGuard</b> | <b>0.9998</b> | <b>0.9988</b> | 0.9973        | <b>0.9980</b> | <b>0.01%</b> |
| Trace      | Threatrace       | 0.9892        | 0.7159        | <b>0.9999</b> | 0.8343        | 1.10%        |
|            | MAGIC            | 0.9969        | 0.9697        | <u>0.9997</u> | 0.9845        | 0.34%        |
|            | FLASH            | <u>0.9975</u> | 0.9466        | <b>0.9999</b> | 0.9720        | 0.16%        |
|            | STGAN            | 0.9991        | <u>0.9950</u> | 0.9883        | <u>0.9916</u> | <b>0.01%</b> |
|            | <b>ProvGuard</b> | <b>0.9998</b> | <b>0.9954</b> | <u>0.9997</u> | <b>0.9966</b> | <b>0.01%</b> |
| Streamspot | Unicorn          | 0.99          | 0.95          | 0.97          | 0.96          | 1.6%         |
|            | Threatrace       | 0.99          | 0.98          | 0.99          | 0.99          | 0.4%         |
|            | MAGIC            | 0.99          | 0.99          | <b>1.00</b>   | 0.99          | <u>0.6%</u>  |
|            | FLASH            | 0.99          | 0.98          | 0.99          | 0.99          | <b>0.4%</b>  |
|            | <b>ProvGuard</b> | <b>0.99</b>   | <b>0.99</b>   | <b>1.00</b>   | <b>0.99</b>   | <u>0.6%</u>  |
| Wget       | Unicorn          | 0.90          | 0.86          | 0.95          | 0.90          | 15.5%        |
|            | Threatrace       | 0.95          | <u>0.93</u>   | <u>0.98</u>   | <u>0.95</u>   | 7.4%         |
|            | MAGIC            | 0.94          | <b>0.96</b>   | 0.92          | 0.94          | <u>2.0%</u>  |
|            | FLASH            | <u>0.96</u>   | <b>0.96</b>   | 0.96          | <b>0.96</b>   | 4.0%         |
|            | <b>ProvGuard</b> | <b>0.98</b>   | <b>0.98</b>   | <b>0.96</b>   | <b>0.96</b>   | <b>0.6%</b>  |

For instance, under CGPA-50%, ProvGuard achieves an F1 score of 0.872 and an AUC of 0.975, significantly surpassing other methods. Its strength lies in guiding the model to learn globally robust representations, thereby mitigating the impact of structural manipulations. Furthermore, we evaluated the robustness of all models under increasing attack ratios (10%, 20%, 50%). The results show that although all methods experience some performance drop under stronger perturbations, ProvGuard exhibits the least degradation, consistently maintaining AUC >0.96 and F1 >0.87 in nearly all cases. It is also important to note that large-scale perturbations are often difficult to execute stealthily in real-world scenarios and tend to leave more forensic traces.

**Visualization of Representations.** The key strength of ProvGuard is its ability to learn high-quality representations that capture global system behavior and form a clear decision boundary, enabling precise separation of benign and malicious nodes. To examine these representations, we visualized the latent space under a graph-manipulation attack (CGPA, attack ratio 0.2) using t-SNE to project each sample’s graph embedding into two dimensions. Figure 4 shows the results: subfigures (a)–(c) depict the feature distribution without attack, while subfigures (d)–(f) show the distribution after the attack. Blue points denote benign samples and red points malicious ones. In the DARPA dataset, where node counts are large, K-means cluster centroids approximate the benign distribution; nodes far from all centroids are considered malicious. ProvGuard’s latent space remains well structured, clearly separating benign and malicious nodes. This compact representation and sharp decision boundary not only improve classification accuracy but also raise the bar for adversarial manipulation, enhancing robustness. Even after

**Table 3: Multi-various Graph Manipulation Attacks under Different Attack Rates. (in Cadets Dataset)**

| Attack | Rate(%) | Threatrace [55] |              | MAGIC [26]   |              | FLASH [43]   |              | STGAN [44]   |              | ProvGuard            |                      |
|--------|---------|-----------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|----------------------|----------------------|
|        |         | F1-Score        | AUC          | F1-Score     | AUC          | F1-Score     | AUC          | F1-Score     | AUC          | F1-Score             | AUC                  |
| None   | \       | 0.949           | 0.954        | 0.970        | 0.997        | 0.966        | 0.978        | 0.998        | 0.998        | <b>0.998</b>         | <b>0.999</b>         |
| GSPA   | 10      | 0.813(-0.14)    | 0.910(-0.04) | 0.848(-0.12) | 0.921(-0.08) | 0.837(-0.13) | 0.921(-0.06) | 0.905(-0.09) | 0.972(-0.03) | <b>0.942</b> (-0.06) | <b>0.996</b> (-0.00) |
|        | 20      | 0.749(-0.20)    | 0.854(-0.10) | 0.800(-0.17) | 0.862(-0.14) | 0.759(-0.21) | 0.889(-0.09) | 0.866(-0.13) | 0.951(-0.05) | <b>0.932</b> (-0.07) | <b>0.984</b> (-0.02) |
|        | 50      | 0.489(-0.46)    | 0.756(-0.20) | 0.533(-0.44) | 0.745(-0.25) | 0.657(-0.31) | 0.828(-0.15) | 0.801(-0.20) | 0.927(-0.07) | <b>0.887</b> (-0.11) | <b>0.972</b> (-0.03) |
| GFPA   | 10      | 0.878(-0.07)    | 0.913(-0.04) | 0.959(-0.01) | 0.991(-0.01) | 0.922(-0.04) | 0.952(-0.03) | 0.968(-0.03) | 0.987(-0.01) | <b>0.988</b> (-0.01) | <b>0.999</b> (0.00)  |
|        | 20      | 0.843(-0.11)    | 0.907(-0.05) | 0.950(-0.02) | 0.979(-0.02) | 0.896(-0.07) | 0.941(-0.04) | 0.957(-0.04) | 0.981(-0.02) | <b>0.976</b> (-0.02) | <b>0.998</b> (-0.00) |
|        | 50      | 0.797(-0.15)    | 0.871(-0.08) | 0.920(-0.05) | 0.957(-0.04) | 0.871(-0.10) | 0.938(-0.04) | 0.931(-0.07) | 0.963(-0.04) | <b>0.963</b> (-0.04) | <b>0.988</b> (-0.01) |
| CGPA   | 10      | 0.845(-0.10)    | 0.901(-0.05) | 0.870(-0.10) | 0.974(-0.02) | 0.877(-0.09) | 0.934(-0.04) | 0.938(-0.06) | 0.985(-0.01) | <b>0.961</b> (-0.03) | <b>0.997</b> (-0.00) |
|        | 20      | 0.749(-0.20)    | 0.882(-0.07) | 0.830(-0.14) | 0.913(-0.08) | 0.841(-0.13) | 0.913(-0.07) | 0.915(-0.08) | 0.972(-0.03) | <b>0.937</b> (-0.06) | <b>0.989</b> (-0.01) |
|        | 50      | 0.489(-0.46)    | 0.824(-0.13) | 0.655(-0.32) | 0.819(-0.18) | 0.777(-0.19) | 0.865(-0.11) | 0.872(-0.13) | 0.952(-0.05) | <b>0.872</b> (-0.13) | <b>0.975</b> (-0.02) |
| SPA    | 10      | 0.803(-0.15)    | 0.882(-0.07) | 0.807(-0.16) | 0.958(-0.04) | 0.808(-0.16) | 0.895(-0.08) | 0.942(-0.06) | 0.982(-0.02) | <b>0.983</b> (-0.01) | <b>0.995</b> (-0.00) |
|        | 20      | 0.739(-0.21)    | 0.842(-0.11) | 0.740(-0.23) | 0.873(-0.12) | 0.668(-0.30) | 0.830(-0.15) | 0.902(-0.10) | 0.964(-0.03) | <b>0.937</b> (-0.06) | <b>0.983</b> (-0.02) |
|        | 50      | 0.589(-0.36)    | 0.761(-0.19) | 0.631(-0.34) | 0.815(-0.18) | 0.522(-0.44) | 0.753(-0.23) | 0.834(-0.16) | 0.936(-0.06) | <b>0.899</b> (-0.10) | <b>0.962</b> (-0.04) |
| FPA    | 10      | 0.821(-0.13)    | 0.904(-0.05) | 0.960(-0.01) | 0.983(-0.01) | 0.952(-0.01) | 0.953(-0.03) | 0.972(-0.03) | 0.989(-0.01) | <b>0.989</b> (-0.01) | <b>0.999</b> (0.00)  |
|        | 20      | 0.777(-0.17)    | 0.895(-0.06) | 0.940(-0.03) | 0.953(-0.04) | 0.946(-0.02) | 0.948(-0.03) | 0.962(-0.04) | 0.983(-0.02) | <b>0.970</b> (-0.03) | <b>0.998</b> (-0.00) |
|        | 50      | 0.759(-0.19)    | 0.874(-0.08) | 0.890(-0.08) | 0.916(-0.08) | 0.931(-0.04) | 0.922(-0.06) | 0.938(-0.06) | 0.974(-0.02) | <b>0.951</b> (-0.05) | <b>0.981</b> (-0.02) |

**Figure 4: Latent representations learned by ProvGuard before and after graph manipulation attacks.**

the attack, only minor dispersion appears and ProvGuard continues to distinguish classes effectively. This resilience stems from its contrastive learning objective, which pulls together samples from different augmented views and drives the model to focus on global rather than local features, leaving graph-manipulation attacks with limited impact.

#### 4.4 Ablation Study

To rigorously evaluate the contribution of the Graph Contrastive Learning (GCL) module in ProvGuard, we conducted an ablation study on the Cadets dataset under a CGPA attack with a 20% perturbation ratio. In this experiment, we removed the GCL component and replaced it with alternative graph representation methods, including GraphSAGE [17], DGI [50], and MGAE [49]. We also disabled the Logic-Aware Noise Injection (LIN), adopting purely random graph augmentations to assess its effect, while keeping

the K-means-based detector and identical training–testing splits unchanged.

The results are reported in Table 4. The complete ProvGuard model achieves the best performance in both clean and adversarial settings, reaching F1/AUC of 0.998/0.999 on clean data and maintaining 0.937/0.989 under CGPA attacks with only negligible degradation. Removing the GCL module causes the most pronounced loss of robustness (F1 drop of 0.32 and AUC drop of 0.11), demonstrating that contrastive learning is essential for maintaining a stable decision boundary. Replacing GCL with DGI or GraphSAGE further reduces precision and AUC, underscoring the advantage of ProvGuard’s multi-view contrastive objective. Among the alternatives, MGAE exhibits the second-best robustness (F1 = 0.890, AUC = 0.951), likely because its masked reconstruction mechanism can partially hide perturbed nodes during training and thereby preserve global structure. In addition, the ProvGuard variant with random augmentation (w/o LIN) still delivers competitive performance, but its robustness against adversarial attacks is noticeably weaker. This indicates that employing logically constrained augmentation yields stronger defenses and highlights the importance of LIN in achieving superior robustness.

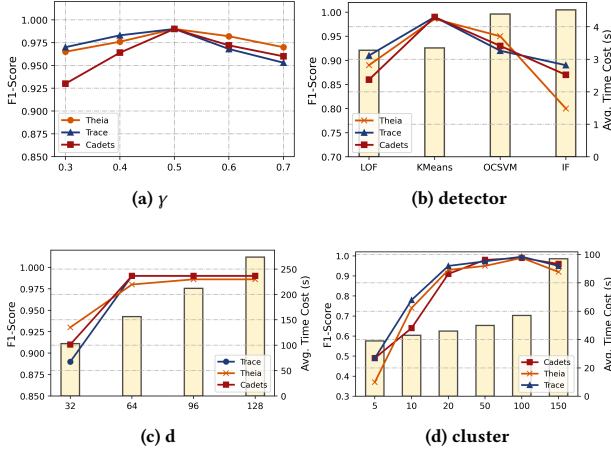
#### 4.5 Parameter Sensitive Analysis

To determine the optimal parameter in our approach, we conducted a hyperparameter search over a range of candidate values. The selection was guided by actual performance metrics, including average time cost and F1 score.

**Augmentation Ratio ( $\gamma$ ).** The augmentation ratio controls the degree of graph modification during training. Figure 5(a) shows that the augmentation ratio has a minimal impact on ProvGuard’s performance. However, higher augmentation ratios increase training time due to the added complexity in data processing. Additionally, the augmentation ratio also affects ProvGuard’s robustness. Therefore, we recommend selecting a ratio of 0.5 to achieve optimal detection performance and robustness.

**Table 4: Ablation study on graph representation model. (Bold denotes the best result, and underlined denotes the second-best result.)**

| Models                 | None Attack  |              |              |              |              | CGPA ( $\gamma=20\%$ ) |                      |                      |
|------------------------|--------------|--------------|--------------|--------------|--------------|------------------------|----------------------|----------------------|
|                        | Accuracy     | Precision    | Recall       | F1-Score     | AUC          | Precision              | F1-Score             | AUC                  |
| ProvGuard w/ GraphSAGE | <u>0.999</u> | 0.921        | 0.967        | 0.920        | 0.940        | 0.379(-0.54)           | 0.597(-0.32)         | 0.831(-0.11)         |
| ProvGuard w/ DGI       | 0.994        | 0.903        | 0.801        | 0.911        | 0.904        | 0.461(-0.44)           | 0.638(-0.27)         | 0.836(-0.07)         |
| ProvGuard w/ MGAE      | 0.991        | <u>0.953</u> | 0.922        | 0.940        | 0.950        | 0.891(-0.06)           | 0.890(-0.05)         | 0.951(-0.00)         |
| ProvGuard w/o LIN      | <u>0.999</u> | 0.991        | <u>0.985</u> | <u>0.989</u> | <u>0.988</u> | <u>0.916</u> (-0.08)   | <u>0.910</u> (-0.08) | <u>0.964</u> (-0.02) |
| ProvGuard (full)       | <b>0.999</b> | <b>0.998</b> | <b>0.997</b> | <b>0.998</b> | <b>0.999</b> | <b>0.953</b> (-0.05)   | <b>0.937</b> (-0.06) | <b>0.989</b> (-0.01) |

**Figure 5: (a),(b),(c),(d) show the result of augmentation ratio  $\gamma$ , different detector selection, embedding dimension  $d$ , and number of clusters  $k$ , respectively.**

**Effective of Detector.** We explored the classification performance of ProvGuard by employing different classifiers. To this end, we configured several lightweight classifiers, including Local Outlier Factor (LOF) [9], One-Class Support Vector Machine (OCSVM) [47], and Isolation Forest (IF) [33]. We conducted experiments with multiple detectors on the Cadets dataset, and the results are shown in Figure 5.(b). The experimental results show that IF and OCSVM incurred relatively high time overhead and delivered moderate performance. In contrast, LOF had a similar time cost to KMeans but exhibited slightly lower accuracy. Overall, KMeans achieved the best classification performance while maintaining low time overhead.

**Embedding Dimensions ( $d$ ).** Embedding dimensions determine the richness of node representations in the graph. Figure 5(c) shows that higher embedding dimensions do not significantly improve performance but increase computational costs. ProvGuard’s 64 embedding dimension effectively captures graph features even at lower dimensions, striking a balance between performance and efficiency.

**Cluster number  $k$ .** The cluster number primarily affects how well the generated clusters can capture the distribution of benign nodes. Our results are shown in Figure 5.(d). The results indicate that as the number of clusters  $k$  increases, the quality of the cluster centers improves, leading to an increase in the F1 score. However, this also results in a corresponding increase in inference time. By leveraging

cluster centers computed via the K-means algorithm instead of performing KNN neighbor search, we significantly accelerate the detection process.

## 5 Related Work

**Provenance-based Intrusion Detection.** Provenance-based intrusion detection systems (PIDS) can be broadly divided into knowledge-driven and learning-based approaches [12, 25, 64].

Knowledge-driven methods rely on predefined rules or labels to match anomalous patterns in streaming provenance graphs. Representative systems such as SLEUTH [24], HOLMES [38], RapSheet [20], APTShield [63], ProTagging [45], and CAPTION [51] encode tactics, techniques, and procedures (TTPs) for incremental pattern matching, whereas Poirot [37] performs template-based graph matching. Although efficient for known attack signatures, these methods depend heavily on expert knowledge and cannot identify novel attack strategies.

Learning-based approaches instead infer benign system behavior from historical audit logs and flag nodes or subgraphs that deviate from these patterns during inference. Prior studies address multiple granularities, including node, subgraph, and path/edge detection as it provides finer-grained analysis and lower false-positive rates. Representative methods include Threatrace [55], MAGIC [26], FLASH [43], KAIROS [10], ORTHRUS [27], TAPAS [62], Vodka [56], STGAN [44], and Velox [8], which adopt techniques such as graph autoencoders, temporal graph networks, spatio-temporal attention, and knowledge distillation to improve scalability, efficiency, and detection accuracy.

## 6 Conclusion

We introduced ProvGuard, a novel PIDS designed to enhance the robustness of provenance-based intrusion detection. By integrating multi-view augmentations with contrastive learning, ProvGuard effectively mitigates mimicry attacks that manipulate graph structures. Comprehensive evaluations on multiple datasets demonstrate that ProvGuard outperforms state-of-the-art detectors in both robustness and detection accuracy, without compromising efficiency.

## Acknowledgments

This research was supported in part by the National Natural Science Foundation of China (625B2139, 62572377, 62302362) and Shaanxi Province Natural Science Basic Research Program (Project No. 2025SYS-SYSZD-081)

## References

- [1] [n. d.]. Apt notes. <https://github.com/kbandla/APTnotes>, Last accessed on 2025-7-25.
- [2] [n. d.]. Streamspot Dataset. <https://github.com/sbustreamspot/sbustreamspot-data>, Last accessed on 2025-7-29.
- [3] [n. d.]. Transparent Computing Engagement 3 DataRelease. <https://github.com/darpa-i2o/TransparentComputing/blob/master/README-E3.md>, Last accessed on 2025-7-21.
- [4] Abdullah Al-Dujaili, Alex Huang, Erik Hemberg, and Una-May O'Reilly. 2018. Adversarial Deep Learning for Robust Detection of Binary Encoded Malware. In *2018 IEEE Symposium on Security and Privacy Workshops (SPW)*. IEEE, 76–82.
- [5] Abdulellah Alsaheel, Yuhong Nan, Shiqing Ma, Le Yu, Gregory Walkup, Z Berkay Celik, Xiangyu Zhang, and Dongyan Xu. 2021. ATLAS: A Sequence-based Learning Approach for Attack Investigation.. In *USENIX Security Symposium*. 3005–3022.
- [6] Anish Athalye, Nicholas Carlini, and David Wagner. 2018. Obfuscated Gradients Give a False Sense of Security: Circumventing Defenses to Adversarial Examples. In *Proceedings of the 35th International Conference on Machine Learning*. PMLR, 274–283.
- [7] Adam Bates, Dave Jing Tian, Kevin RB Butler, and Thomas Moyer. 2015. Trustworthy {Whole-System} provenance for the linux kernel. In *24th USENIX Security Symposium (USENIX Security 15)*. 319–334.
- [8] Tristan Bilot, Baoxiang Jiang, Zefeng Li, Nour El Madhoun, Khaldoun Al Agha, Anis Zouaoui, and Thomas Pasquier. 2025. Sometimes Simpler is Better: A Comprehensive Analysis of {State-of-the-Art} {Provenance-Based} Intrusion Detection Systems. In *34th USENIX Security Symposium (USENIX Security 25)*. 7193–7212.
- [9] Markus M Breunig, Hans-Peter Kriegel, Raymond T Ng, and Jörg Sander. 2000. LOF: identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*. 93–104.
- [10] Zijun Cheng, Qiujian Lv, Jinyuan Liang, Yan Wang, Degang Sun, Thomas Pasquier, and Xueyuan Han. 2024. KAIROS: Practical intrusion detection and investigation using whole-system provenance. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 3533–3551.
- [11] Francesco Croce and Matthias Hein. 2020. Reliable Evaluation of Adversarial Robustness with an Ensemble of Diverse Parameter-Free Attacks. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*. PMLR, 2206–2216.
- [12] Feng Dong, Shaofei Li, Peng Jiang, Ding Li, Haoyu Wang, Liangyi Huang, Xusheng Xiao, Jiedong Chen, Xiapu Luo, Yao Guo, et al. 2023. Are we there yet? an industrial viewpoint on provenance-based endpoint detection and response tools. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2396–2410.
- [13] Feng Dong, Liu Wang, Xu Nie, Fei Shao, Haoyu Wang, Ding Li, Xiapu Luo, and Xusheng Xiao. 2023. {DISTDET}: A {Cost-Effective} Distributed Cyber Threat Detection System. In *32nd USENIX Security Symposium (USENIX Security 23)*. 6575–6592.
- [14] Pengcheng Fang, Peng Gao, Changlin Liu, Erman Ayday, Kangkook Jee, Ting Wang, Yanfang Fanny Ye, Zhuotao Liu, and Xusheng Xiao. 2022. {Back-Propagating} System Dependency Impact for Attack Investigation. In *31st USENIX Security Symposium (USENIX Security 22)*. 2461–2478.
- [15] Akul Goyal, Xueyuan Han, Gang Wang, and Adam Bates. 2023. Sometimes, You Aren't What You Do: Mimicry Attacks against Provenance Graph Host Intrusion Detection Systems. In *30th Network and Distributed System Security Symposium*.
- [16] Gongde Guo, Hui Wang, David Bell, Yaxin Bi, and Kieran Greer. 2003. KNN model-based approach in classification. In *On The Move to Meaningful Internet Systems 2003: CoopIS, DOA, and ODBASE: OTM Confederated International Conferences, CoopIS, DOA, and ODBASE 2003, Catania, Sicily, Italy, November 3-7, 2003*. Proceedings. Springer, 986–996.
- [17] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).
- [18] Xueyuan Han, Thomas Pasquier, Adam Bates, James Mickens, and Margo Seltzer. 2020. Unicorn: Runtime provenance-based detector for advanced persistent threats.. In *NDSS*.
- [19] Xueyuan Han, Xiao Yu, Thomas Pasquier, Ding Li, Junghwan Rhee, James Mickens, Margo Seltzer, and Haifeng Chen. 2021. SIGL: Securing software installations through deep graph learning. In *30th USENIX Security Symposium (USENIX Security 21)*. 2345–2362.
- [20] Wajih Ul Hassan, Adam Bates, and Daniel Marino. 2020. Tactical provenance analysis for endpoint detection and response systems. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1172–1189.
- [21] Wajih Ul Hassan, Shengjian Guo, Ding Li, Zhengzhang Chen, Kangkook Jee, Zhichun Li, and Adam Bates. 2019. Nodoe: Combatting threat alert fatigue with automated provenance triage. In *network and distributed systems security symposium*.
- [22] Wajih Ul Hassan, Mohammad Ali Nouredine, Pubali Datta, and Adam Bates. 2020. OmegaLog: High-fidelity attack investigation via transparent multi-layer log analysis. In *Network and distributed system security symposium*.
- [23] Kaveh Hassani and Amir Hosein Khasahmadi. 2020. Contrastive multi-view representation learning on graphs. In *International conference on machine learning*. PMLR, 4116–4126.
- [24] Md Nahid Hossain, Sadegh M Milajerdi, Junao Wang, Birhanu Eshete, Rigel Gjomemo, R Sekar, Scott D Stoller, and VN Venkatakrishnan. 2017. SLEUTH: Real-time Attack Scenario Reconstruction from COTS Audit Data.. In *USENIX Security Symposium*. 487–504.
- [25] Muhammad Adil Inam, Yinfang Chen, Akul Goyal, Jason Liu, Jaron Mink, Noor Michael, Sneha Gaur, Adam Bates, and Wajih Ul Hassan. 2022. SoK: History is a Vast Early Warning System: Auditing the Provenance of System Intrusions. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 307–325.
- [26] Zian Jia, Yun Xiong, Yuhong Nan, Yao Zhang, Jinjing Zhao, and Mi Wen. 2024. MAGIC: Detecting Advanced Persistent Threats via Masked Graph Representation Learning. In *33rd USENIX Security Symposium (USENIX Security 24)*. 5197–5214.
- [27] Baoxiang Jiang, Tristan Bilot, Nour El Madhoun, Khaldoun Al Agha, Anis Zouaoui, Shahrear Iqbal, Xueyuan Han, and Thomas Pasquier. 2025. ORTHUS: Achieving High Quality of Attribution in Provenance-based Intrusion Detection Systems. In *Security Symposium (USENIX Sec'25)*. USENIX.
- [28] Samuel T King and Peter M Chen. 2003. Backtracking intrusions. In *Proceedings of the nineteenth ACM symposium on Operating systems principles*. 223–236.
- [29] Samuel T King, Zhuoqing Morley Mao, Dominic G Luchetti, and Peter M Chen. 2005. Enriching Intrusion Alerts Through Multi-Host Causality.. In *Ndss*. Citeseer.
- [30] Yonghui Kwon, Dohyeong Kim, William Nick Sumner, Kyungtae Kim, Brendan Saltaformaggio, Xiangyu Zhang, and Dongyan Xu. 2016. Ldx: Causality inference by lightweight dual execution. In *Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems*. 503–515.
- [31] Yonghui Kwon, Fei Wang, Weihang Wang, Kyu Hyung Lee, Wen-Chuan Lee, Shiqing Ma, Xiangyu Zhang, Dongyan Xu, Somesh Jha, Gabriela F Ciocarlie, et al. 2018. MCI: Modeling-based Causality Inference in Audit Logging for Attack Investigation.. In *NDSS*, Vol. 2. 4.
- [32] Kyu Hyung Lee, Xiangyu Zhang, and Dongyan Xu. 2013. High Accuracy Attack Provenance via Binary-based Execution Partition.. In *NDSS*, Vol. 16.
- [33] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. 2008. Isolation forest. In *2008 eighth IEEE international conference on data mining*. IEEE, 413–422.
- [34] Yushan Liu, Mu Zhang, Ding Li, Kangkook Jee, Zhichun Li, Zhenyu Wu, Junghwan Rhee, and Prateek Mittal. 2018. Towards a Timely Causality Analysis for Enterprise Security.. In *NDSS*.
- [35] Shiqing Ma, Juan Zhai, Fei Wang, Kyu Hyung Lee, Xiangyu Zhang, and Dongyan Xu. 2017. MPI: Multiple Perspective Attack Investigation with Semantic Aware Execution Partitioning.. In *USENIX Security Symposium*. 1111–1128.
- [36] Emaad Manzoor, Sadegh M Milajerdi, and Leman Akoglu. 2016. Fast memory-efficient anomaly detection in streaming heterogeneous graphs. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*. 1035–1044.
- [37] Sadegh M Milajerdi, Birhanu Eshete, Rigel Gjomemo, and VN Venkatakrishnan. 2019. Poirat: Aligning attack behavior with kernel audit records for cyber threat hunting. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*. 1795–1812.
- [38] Sadegh M Milajerdi, Rigel Gjomemo, Birhanu Eshete, Ramachandran Sekar, and VN Venkatakrishnan. 2019. Holmes: real-time apt detection through correlation of suspicious information flows. In *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1137–1152.
- [39] Kunal Mukherjee, Joshua Wiedemeier, Tianhao Wang, James Wei, Feng Chen, Muhyun Kim, Murat Kantarcioglu, and Kangkook Jee. 2023. Evading Provenance-Based ML Detectors with Adversarial System Actions. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 1199–1216. <https://www.usenix.org/conference/usenixsecurity23/presentation/mukherjee>
- [40] Riccardo Paccagnella, Pubali Datta, Wajih Ul Hassan, Adam Bates, Christopher Fletcher, Andrew Miller, and Dave Tian. 2020. Custos: Practical tamper-evident auditing of operating systems using trusted execution. In *Network and distributed system security symposium*.
- [41] Thomas Pasquier, Xueyuan Han, Mark Goldstein, Thomas Moyer, David Eysers, Margo Seltzer, and Jean Bacon. 2017. Practical whole-system provenance capture. In *Proceedings of the 2017 Symposium on Cloud Computing*. 405–418.
- [42] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [43] Mati Ur Rehman, Hadi Ahmadi, and Wajih Ul Hassan. 2024. FLASH: A Comprehensive Approach to Intrusion Detection via Provenance Graph Representation Learning. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 139–139.

- [44] Anyuan Sang, Xuezheng Fan, Li Yang, Yuchen Wang, Lu Zhou, Junbo Jia, and Huipeng Yang. 2025. STGAN: Detecting Host Threats via Fusion of Spatial-Temporal Features in Host Provenance Graphs. In *Proceedings of the ACM on Web Conference 2025*. 1046–1057.
- [45] Anyuan Sang, Yuchen Wang, Junbo Jia, Li Yang, Pengbin Feng, Lu Zhou, and Jianfeng Ma. 2025. Resist Dependency Explosion in Attack Investigation With Splittable Tag Propagation and Aggregation. *IEEE Transactions on Dependable and Secure Computing* (2025).
- [46] Anyuan Sang, Yuchen Wang, Li Yang, Junbo Jia, and Lu Zhou. 2024. Obfuscating Provenance-Based Forensic Investigations with Mapping System Meta-Behavior. In *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses* (Padua, Italy) (RAID '24).
- [47] Bernhard Schölkopf, Robert C Williamson, Alex Smola, John Shawe-Taylor, and John Platt. 1999. Support vector method for novelty detection. *Advances in neural information processing systems* 12 (1999).
- [48] Yun Shen and Gianluca Stringhini. 2019. Attack2vec: Leveraging temporal word embeddings to understand the evolution of cyberattacks. *28 st USENIX Security Symposium (USENIX Security 2019)* (2019).
- [49] Qiaoyu Tan, Ninghao Liu, Xiao Huang, Rui Chen, Soo-Hyun Choi, and Xia Hu. 2022. Mgae: Masked autoencoders for self-supervised learning on graphs. *arXiv preprint arXiv:2201.02534* (2022).
- [50] Petar Velickovic, William Fedus, William L Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. 2019. Deep graph infomax. *ICLR (Poster)* 2, 3 (2019), 4.
- [51] Lingzhi Wang, Xiangmin Shen, Weijian Li, Zhenyuan Li, R Sekar, Han Liu, and Yan Chen. 2024. Incorporating gradients to rules: Towards lightweight, adaptive provenance-based intrusion detection. *arXiv preprint arXiv:2404.14720* (2024).
- [52] Minjie Wang, Da Zheng, Zihao Ye, Quan Gan, Mufei Li, Xiang Song, Jinjing Zhou, Chao Ma, Lingfan Yu, Yu Gai, et al. 2019. Deep graph library: A graph-centric, highly-performant package for graph neural networks. *arXiv preprint arXiv:1909.01315* (2019).
- [53] Qinglong Wang, Wenbo Guo, Kaixuan Zhang, Alexander G. Ororbia, Xinyu Xing, C. Lee Giles, and Xue Liu. 2017. Adversary Resistant Deep Neural Networks with an Application to Malware Detection. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 1145–1153.
- [54] Qi Wang, Wajih Ul Hassan, Ding Li, Kangkook Jee, Xiao Yu, Kexuan Zou, Junghwan Rhee, Zhengzhang Chen, Wei Cheng, Carl A Gunter, et al. 2020. You Are What You Do: Hunting Stealthy Malware via Data Provenance Analysis.. In *NDSS*.
- [55] Su Wang, Zhiliang Wang, Tao Zhou, Hongbin Sun, Xia Yin, Dongqi Han, Han Zhang, Xingang Shi, and Jiahai Yang. 2022. Threatrace: Detecting and tracing host-based threats in node level through provenance graph learning. *IEEE Transactions on Information Forensics and Security* 17 (2022), 3972–3987.
- [56] Weiheng Wu, Wei Qiao, Wenhao Yan, Bo Jiang, Yuling Liu, Baoxu Liu, Zhigang Lu, and Junrong Liu. 2025. Brewing Vodka: Distilling Pure Knowledge for Lightweight Threat Detection in Audit Logs. In *Proceedings of the ACM on Web Conference 2025*. 2172–2182.
- [57] Zhang Xu, Zhenyu Wu, Zhichun Li, Kangkook Jee, Junghwan Rhee, Xusheng Xiao, Fengyuan Xu, Haining Wang, and Guofei Jiang. 2016. High fidelity data reduction for big data security dependency analyses. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. 504–516.
- [58] Fan Yang, Jiace Xu, Chunlin Xiong, Zhou Li, and Kehuan Zhang. 2023. PROGRAPHER: An Anomaly Detection System based on Provenance Graph Embedding. In *32nd USENIX Security Symposium (USENIX Security 23)*. 4355–4372.
- [59] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. 2020. Graph contrastive learning with augmentations. *Advances in neural information processing systems* 33 (2020), 5812–5823.
- [60] Jun Zeng, Xiang Wang, Jiahao Liu, Yinfang Chen, Zhenkai Liang, Tat-Seng Chua, and Zheng Leong Chua. 2022. Shadewatcher: Recommendation-guided cyber threat analysis using system audit records. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 489–506.
- [61] Jun Zeng, Chuqi Zhang, and Zhenkai Liang. 2022. PalanTir: Optimizing Attack Provenance with Hardware-enhanced System Observability. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 3135–3149.
- [62] Bo Zhang, Yansong Gao, Changlong Yu, Boyu Kuang, Zhi Zhang, Hyoungshick Kim, and Anmin Fu. 2025. {TAPAS}: An Efficient Online {APT} Detection with Task-guided Process Provenance Graph Segmentation and Analysis. In *34th USENIX Security Symposium (USENIX Security 25)*. 607–624.
- [63] Tiantian Zhu, Jinkai Yu, Chunlin Xiong, Wenrui Cheng, Qixuan Yuan, Jie Ying, Tieming Chen, Jiabo Zhang, Mingqi Lv, Yan Chen, et al. 2023. APTSHIELD: A Stable, Efficient and Real-time APT Detection System for Linux Hosts. *IEEE Transactions on Dependable and Secure Computing* (2023).
- [64] Michael Zipperle, Florian Gottwalt, Elizabeth Chang, and Tharam Dillon. 2022. Provenance-based intrusion detection systems: A survey. *Comput. Surveys* 55, 7 (2022), 1–36.

## A More Experiments Setup

### A.1 Implementation.

In data processing, we adopted the log transformers in MAGIC [26] for processing streaming audit logs, including StreamSpot [36], Camflow [41], and DARPA TC Dataset [3]. Networkx is used to construct the provenance graph. The graph indicates that the learning module is implemented by Pytorch [42] and DGL [52].

### A.2 Parameter Settings.

For the setup of ProvGuard, the learning rate  $lr$  is set to 0.001. We use a 2-layer GAT encoder, and in data augmentation, the augmentation ratio is set to 0.5. The training batch size is 50 with  $d$  set to 64 on the DARPA TC dataset.

### A.3 Dataset Description.

**Table 5: Dataset Statistics for Streamspot and Unicorn Wget**

| Dataset      | Graph Pieces | Entities | Interactions | Size (GB) |
|--------------|--------------|----------|--------------|-----------|
| Streamspot   | 100          | 8,292    | 113,229      | 2.8       |
|              |              | 8,636    | 112,958      |           |
|              |              | 8,989    | 294,903      |           |
|              |              | 8,830    | 310,814      |           |
|              |              | 6,826    | 37,382       |           |
| Unicorn Wget | 25           | 8,890    | 28,423       | 76        |
|              |              | 125      | 265,424      |           |
|              |              | 25       | 257,156      |           |

**Table 6: Dataset Statistics for DARPA E3**

| Dataset   | Benign Nodes | Abnormal Nodes | Edges      | Size (GB) |
|-----------|--------------|----------------|------------|-----------|
| E3 Trace  | 3,220,596    | 68,082         | 4,080,457  | 67        |
| E3 Cadets | 1,614,189    | 12,846         | 3,303,264  |           |
| E3 Theia  | 3,505,326    | 25,362         | 10,929,710 |           |

**DARPA TC dataset.** The DARPA TC dataset is a benchmark dataset provided by DARPA for evaluating cybersecurity and intrusion detection systems. It was collected from networks during adversarial engagements. The red team conducted APT attacks using various vulnerabilities to exfiltrate information. Our evaluation includes the TRACE, CADETS, and THEIA subdatasets, which contain millions of entities and interaction records. We used the ground truth information provided by the Threatrace [55] to perform entity-level detection and conduct attack investigations.

**Unicorn Wget dataset.** The Wget dataset was designed by the authors of Unicorn [18] to simulate attack scenarios. It uses the Camflow [41] system to collect 150 batches of audit logs, with 125 batches containing no attack processes and 25 batches containing supply chain attacks. These attacks are carefully crafted to mimic benign entity interactions, making this dataset challenging to identify due to its large data volume and stealthy attacks. We will perform graph-level detection on this dataset as in previous approaches.

**StreamSpot dataset.** The StreamSpot dataset is a publicly available dataset provided by the authors of StreamSpot [36], containing 600 information flow graphs. These graphs come from five benign

**Table 7: Details of Graph Manipulation Attacks**

| Phase     | Attack Type | Target      | Rate ( $y$ )  |
|-----------|-------------|-------------|---------------|
| Detection | GSPA        | Node        | $y$           |
|           | GFPA        | Edge        | $y$           |
|           | CGPA        | Node & Edge | $0.5y + 0.5y$ |
| Training  | SPA         | Node & Edge | $0.5y + 0.5y$ |
|           | FPA         | Node        | $y$           |

scenarios and one attack scenario. Each scenario runs 100 times, generating 100 graphs using the Linux SystemTap Logging System. The five benign scenarios simulate normal user behavior, while the attack scenario simulates a drive-by download attack. We performed graph-level anomaly detection on the StreamSpot dataset, similar to previous studies [18, 55], as it only provides graph-level ground truth.

#### A.4 Baseline Methods Description.

We compared STGAN with several baseline methods, including Unicorn, Threatrace [55], MAGIC [26], FLASH [43], and STGAN [44]. More descriptions of these baseline methods as follows.

**Unicorn [18]:** This detector employs sketching techniques, leveraging historical information to model and identify anomalous activities.

**Threatrace [55]:** This detector uses GraphSAGE [17] to learn node embeddings and performs anomaly detection.

**FLASH [43]:** This detector extracts semantic features of nodes using Word2Vec and then applies GAT for structural feature extraction, followed by anomaly detection.

**MAGIC [26]:** MAGIC is an unsupervised anomaly detector based on MGAE [49]. It learns node embeddings through MGAE and then applies a KNN-based outlier detection to identify malicious nodes.

**STGAN [44]:** STGAN is an anomaly detector that fuses spatio-temporal features. It utilizes a Graph Attention Network (GAT) to capture structural features, Word2Vec to extract semantic features, and a Temporal Graph Network (TGN) to model temporal dynamics. After integrating these features, it employs XGBoost for anomaly detection.

#### A.5 Graph Manipulation Attacks.

(1) *Data Pollution Attacks.* Data pollution attacks aim to manipulate graph structures during the detection phase to hide malicious behaviors. For this category, we implemented three types of graph manipulation attacks:

**Graph Feature Pollution Attack (GFPA).** Alters the features of malicious nodes to mimic those of benign nodes, thereby hiding malicious behavior and evading detection.

**Graph Structure Pollution Attack (GSPA).** Selectively adds new edges between malicious nodes and benign nodes, thereby altering the graph structure to make malicious nodes appear similar to benign nodes.

**Combined Graph Pollution Attack (CGPA).** Combines both malicious feature manipulation and malicious structure manipulation methods, simultaneously altering the features and structure of malicious nodes to maximize the concealment of malicious behavior and evade detection.

These attacks were simulated by perturbing malicious nodes and their surrounding structures within the victim graph, mimicking realistic attacker behavior aimed at tampering with the graph.

(2) *Data Poisoning Attacks.* Data poisoning attacks target the training phase, where the attacker perturbs the graph data used for training to compromise the model’s robustness. Considering the practical difficulty for attackers to access the model directly, we focused on two types of poisoning attacks:

**Structure Poisoning Attack (SPA):** Perturbs a certain proportion of nodes and edges in the training graph by adding or modifying connections, thereby disrupting the original structural features.

**Feature Poisoning Attack (FPA):** Alters a certain proportion of node features in the training graph by swapping initial features between nodes, disrupting the feature distribution, and misleading the model during training.

In summary, we provide detailed information about the attacks in Table 7, including the attack targets (nodes or edges) and the perturbation rate ( $y$ ). Specifically, the perturbation rate ( $y$ ) represents the proportion of nodes or edges manipulated within the entire graph structure. These attacks are constructed at the node level.

## B Time Complexity of ProvGuard

We analyze the time complexity of ProvGuard by examining its main computational components. For a provenance graph with  $|V|$  nodes,  $|E|$  edges, hidden dimension  $d$ , attention heads  $k$ ,  $G$  training graphs, and  $T$  training epochs, the graph augmentation step that performs random node or edge addition and deletion requires  $O(|V| + |E|)$  time. Each of the two Graph Attention Network layers aggregates neighbor features with complexity  $O(k|E|d)$ , which dominates when  $|E| \gg |V|$ . Because two augmented views are processed for every graph in each epoch, the encoder cost is doubled but remains in the same order. Consequently, the overall training complexity is  $O(TG(|V| + |E| + k|E|d))$ , which simplifies to  $O(TGk|E|d)$  for large graphs, and the memory usage is  $O(|V|d + |E|d)$ , indicating that ProvGuard scales linearly with the number of graphs, edges, and training epochs.

## C Pseudo-codes of ProvGuard Detection

Algorithm 1 describes the anomaly detection procedure using graph embeddings learned by ProvGuard. Given embeddings of benign training graphs, a distance-based K-means scoring function is applied to test samples. Higher average distances indicate higher anomaly scores.

## D Discussion

**Adaptive Attack Analysis.** While ProvGuard demonstrates strong robustness against known graph manipulation attacks, we recognize that adversaries may adapt once the defense strategy is revealed. An attacker could attempt to craft malicious behaviors that imitate the statistics of our logic-aware augmentations. ProvGuard mitigates these risks by learning a compact manifold of benign behaviors through contrastive learning and by capturing contextual and causal semantics rather than relying on explicit structural inconsistencies. Any provenance graph that deviates from the learned semantic distribution can still be detected as abnormal. Fully adaptive adversaries remain a potential threat and will be addressed

**Algorithm 1** Training and Detection Framework of ProvGuard

---

**Require:** Training graphs  $\mathcal{G}_{train}$ , test graphs  $\mathcal{G}_{test}$ , encoder  $f_\theta$ , projection head  $h_\phi$ , epochs  $T$

**Ensure:** Detected malicious nodes or graphs

```

1: for  $t \leftarrow 1$  to  $T$  do
2:   for each graph  $g \in \mathcal{G}_{train}$  do
3:     Generate two augmented views  $g_1, g_2$ 
4:     Encode:  $z_1 \leftarrow f_\theta(g_1)$ ,  $z_2 \leftarrow f_\theta(g_2)$ 
5:     Project:  $p_1 \leftarrow h_\phi(z_1)$ ,  $p_2 \leftarrow h_\phi(z_2)$ 
6:     Optimize contrastive loss  $\mathcal{L}_{cont}(p_1, p_2)$ 
7:   end for
8: end for
9: Extract embeddings for  $\mathcal{G}_{train}$  and  $\mathcal{G}_{test}$  using  $f_\theta$ 
10: Normalize embeddings and fit K-means on  $\mathcal{G}_{train}$ 
11: For each test sample, compute anomaly score via K-means distance
12: Rank or classify nodes/graphs based on scores
13: return Detected malicious nodes or graphs

```

---

in future work through online model refinement and adversarial training.

**Graph manipulation attacks.** Recently, graph manipulation attacks have posed significant challenges to the performance of provenance-based detectors [15, 39, 46]. Attackers maliciously alter graph structures, causing the graph encoding of malicious nodes to resemble that of normal nodes, leading to false negatives. ProvGuard demonstrates notable advantages in addressing these challenges, primarily due to our innovative approach of introducing various types of perturbations during the training phase and generating embeddings using contrastive learning. This method takes into account the potential manipulations of the graph structure by attackers during training, rendering attempts to alter the structure and features of malicious nodes ineffective. Therefore, ProvGuard suggests a promising approach for countering such graph manipulation attacks in the future.

## E Different Types of Graph Augmentation.

**Table 8: F1 for Different Augmentation Methods under Various Attack Types ( $\gamma_{EA}$ ,  $\gamma_{NA}$ ,  $\gamma_{FA}$ )**

| Augmentation Type | AUC (CGPA)   | AUC (GSPA)   | AUC (GFPA)   | AUC (None)   |
|-------------------|--------------|--------------|--------------|--------------|
| NA                | 0.954        | <b>0.971</b> | 0.967        | <u>0.989</u> |
| EA                | 0.972        | 0.961        | 0.964        | 0.985        |
| FA                | 0.965        | <u>0.963</u> | 0.947        | 0.973        |
| NA+FA             | <u>0.979</u> | <b>0.971</b> | <b>0.988</b> | 0.991        |
| EA+FA             | <b>0.984</b> | 0.959        | <u>0.987</u> | <u>0.989</u> |
| NA+EA+FA          | <b>0.984</b> | <b>0.971</b> | 0.984        | <b>0.999</b> |

ProvGuard introduces three types of graph augmentation methods. These techniques, through contrastive learning, encourage the model to focus on global behaviors while ignoring local perturbations. To investigate the impact of these graph augmentation strategies on the robustness of ProvGuard, we evaluated their effectiveness under a graph contrastive learning framework against different attacks, including GSPA, GFPA, and CGPA. Specifically, we trained the model using various augmentation strategies on the Cadets dataset and assessed its robustness. The experimental results are shown in Table 8. The results show that in the absence of attacks, applying all three augmentation methods achieves the best performance. Under attack scenarios, different augmentation strategies contribute differently to robustness. For example, NA achieves the highest AUC of 0.971 under GSPA attacks, while the combination of NA and FA performs best under GFPA attacks, achieving an AUC of 0.988. The combination of NA, EA, and FA demonstrates consistent robustness across all attack types, with AUC values ranging from 0.971 to 0.984. Overall, Table 8 indicates that well-designed augmentation strategies can effectively enhance ProvGuard’s robustness against various attacks. It is worth noting that although the NA+EA+FA combination does not always yield the best result, possibly due to excessive perturbations interfering with feature extraction, selecting appropriate augmentation strategies can still significantly improve adversarial robustness.

**Table 9: Performance Comparison of Different Methods in Terms of Training Time and Memory Usage**

| Phase            | Metric      | STGAN      | FLASH        | MAGIC      | Threatrace   | ProvGuard    |
|------------------|-------------|------------|--------------|------------|--------------|--------------|
| <b>Train</b>     | Time (s)    | 2,430      | 4,580        | <u>151</u> | 2,780        | <b>214</b>   |
|                  | Memory (MB) | 2,130      | <b>760</b>   | 1,564      | <u>1,031</u> | 1,525        |
| <b>Inference</b> | Times (s)   | <u>733</u> | 4,304        | 1,037      | 1,380        | <b>437</b>   |
|                  | Memory (MB) | 2,011      | <b>1,097</b> | 1,667      | 2,301        | <u>1,532</u> |

## F System Overhead

In this section, we compare the training and inference costs of ProvGuard with its baseline models to evaluate its efficiency. It is important to emphasize that during the training of these detectors, we used the default settings provided in their open-source implementations to achieve optimal performance, and the training process was conducted on the same server and configuration.

Table 9 summarizes the training costs of ProvGuard and its baseline models. We observed that when trained with the same batch size, ProvGuard and MAGIC exhibit similar time and memory overheads. On the other hand, FLASH, STGAN, and Threatrace adopt strategies that allow them to converge with smaller batch sizes and fixed graph sizes within each batch, which reduces memory overhead but results in longer training times. Regarding inference overhead, ProvGuard maintains the best inference time and relatively low memory consumption. Therefore, overall, compared to the training costs of baseline detectors, it can be concluded that ProvGuard ensures robustness without sacrificing efficiency.